Tema 1 - Control de versiones

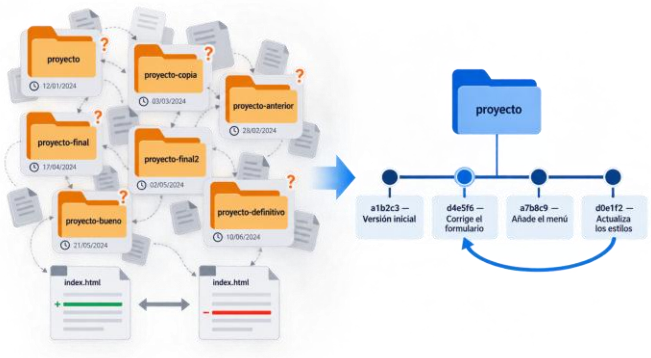
1.1 - Control de versiones con Git y GitHub



Cuando las copias dejan de ser suficientes

Un proyecto informático rara vez permanece invariable. El código se corrige, se añaden funciones, se actualizan dependencias, se modifican archivos de configuración y se eliminan componentes que han dejado de ser necesarios. Cada cambio transforma el estado del proyecto y puede afectar a su funcionamiento.

Una forma elemental de conservar estados anteriores consiste en duplicar periódicamente la carpeta del proyecto. Al principio aparecen nombres como `proyecto`, `proyecto-copia` o `proyecto-anterior`. Más adelante surgen variantes como `proyecto-final`, `proyecto-final2`, `proyecto-bueno` y `proyecto-definitivo`. Estos nombres no describen con precisión qué contiene cada copia ni cómo se relaciona con las demás.



El problema no es únicamente encontrar la carpeta más reciente. También resulta necesario responder a otras preguntas: ¿qué archivos cambiaron?, ¿qué líneas se modificaron?, ¿por qué se realizó cada cambio?, ¿qué versión introdujo un error?, ¿puede recuperarse un archivo anterior sin eliminar el trabajo posterior?

La situación se complica cuando varias personas trabajan sobre el mismo proyecto. Una copia puede contener una función nueva, mientras otra incorpora una corrección distinta. Sustituir una carpeta por otra puede hacer desaparecer uno de esos cambios. Comparar manualmente todos los archivos tampoco garantiza que se detecten las diferencias relevantes.

Definición: Un sistema de control de versiones es una herramienta que registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo y permite consultar, comparar y recuperar estados concretos.

El control de versiones sustituye las copias arbitrarias por un historial organizado. Cada estado registrado queda identificado y relacionado con los anteriores. De este modo, la evolución del proyecto puede comprenderse y gestionarse sin depender de nombres de carpetas ambiguos.

Importante: El control de versiones no sustituye por sí solo a una política de copias de seguridad. Gestiona la evolución lógica de los archivos, pero la protección frente a la pérdida de dispositivos también requiere repositorios remotos y medidas de respaldo.

Qué aporta el control de versiones

El valor de un sistema de control de versiones no se limita a recuperar archivos borrados. Su principal aportación consiste en transformar la evolución de un proyecto en información estructurada y consultable.

Trazabilidad

Cada estado registrado puede incluir información sobre la autoría, la fecha y el propósito del cambio. Esto permite reconstruir cómo se alcanzó la versión actual.

Comparación

Es posible localizar las líneas añadidas, modificadas o eliminadas entre dos estados sin revisar manualmente los archivos completos.

Recuperación

Un archivo o un proyecto pueden devolverse a un estado anterior cuando una modificación produce un resultado incorrecto.

Experimentación

Los cambios pueden desarrollarse de forma aislada y comprobarse antes de incorporarlos a la línea principal.

Colaboración

Varias personas pueden trabajar sobre copias completas del repositorio e integrar posteriormente sus aportaciones.

Revisión

Antes de incorporar un cambio, es posible examinar las diferencias, comentarlas y solicitar correcciones.

Automatización

Los cambios registrados pueden activar pruebas, generación de documentación y procesos de despliegue.

Caso AulaDeploy

A lo largo del capítulo se utilizará un proyecto web denominado AulaDeploy. Su estructura inicial es:

```
auladeploy/
├── index.html
├── css/
│   └── estilos.css
├── js/
│   └── app.js
├── config/
│   └── app.example.json
└── README.md
```

El proyecto evolucionará desde una carpeta local hasta un repositorio compartido. Se registrarán cambios, se crearán ramas, se conectará un repositorio remoto y se integrarán aportaciones mediante un proceso de revisión.

Al concluir el recorrido será posible explicar la finalidad del control de versiones, diferenciar Git de GitHub, registrar y consultar cambios, trabajar con ramas, resolver conflictos, sincronizar repositorios y proteger el código y las credenciales.

Sistemas locales, centralizados y distribuidos

Los sistemas de control de versiones pueden organizarse de distintas maneras según el lugar donde almacenan el historial y la relación que mantienen las copias de trabajo.

Sistemas locales

El historial se almacena en el mismo equipo en el que se editan los archivos. Permiten registrar versiones sin depender de una red, pero concentran el proyecto y su evolución en un único dispositivo.

Sistemas centralizados

Un servidor central conserva el repositorio. Los equipos clientes obtienen de él los archivos y envían sus cambios. Este modelo facilita la administración, pero convierte el servidor en un punto crítico. Si deja de estar disponible, determinadas operaciones de colaboración tampoco pueden realizarse.

Sistemas distribuidos

Cada copia del repositorio contiene los archivos y el historial. Las operaciones de consulta y registro pueden realizarse localmente. Posteriormente, los repositorios intercambian cambios.

Git es un sistema distribuido. Al clonar un repositorio no se descarga únicamente la versión actual de los archivos: también se obtiene su historial. Por eso pueden crearse confirmaciones, consultar versiones y trabajar con ramas sin conexión permanente al servidor remoto.



Aspecto	Local	Centralizado	Distribuido
Historial completo en el equipo	Sí	No necesariamente	Sí
Confirmaciones sin conexión	Sí	Depende del sistema	Sí
Servidor necesario para registrar cambios	No	Habitualmente sí	No
Colaboración remota	Muy limitada	Sí	Sí
Ejemplo representativo	RCS	Subversion	Git

Importante: Distribuido no significa desorganizado. Puede existir un repositorio remoto principal, acompañado de reglas de revisión, permisos y una estrategia común de ramas.

Git y GitHub no son lo mismo

Git y GitHub suelen formar parte del mismo flujo de trabajo, pero son herramientas diferentes.

Git

Es un sistema de control de versiones distribuido. Se instala en el equipo y permite crear repositorios, preparar archivos, registrar cambios, consultar el historial, crear ramas, fusionarlas y comunicarse con otros repositorios. La mayoría de las operaciones locales pueden realizarse sin conexión a Internet.

Git gestiona la historia del proyecto.

GitHub

Es una plataforma que aloja repositorios Git y añade servicios de colaboración: gestión de permisos, incidencias, solicitudes de incorporación, revisión de cambios, protección de ramas, automatización y publicación de documentación, entre otros.

GitHub permite alojar, compartir, revisar y coordinar esa historia.

Un repositorio Git puede existir sin GitHub. También puede alojarse en otra plataforma o en un servidor propio. Del mismo modo, una incidencia o una regla de protección de ramas pertenece a GitHub, no a las órdenes internas de Git.



Antes de continuar: Si GitHub deja de estar disponible temporalmente, Git puede seguir registrando confirmaciones en el repositorio local. La conexión será necesaria posteriormente para intercambiar esos cambios con el remoto.

Operación	Git	GitHub
Crear una confirmación local	Sí	No
Consultar el historial local	Sí	Puede representarlo
Crear una rama local	Sí	Puede mostrar ramas publicadas
Alojar el repositorio en Internet	No por sí solo	Sí
Gestionar incidencias	No	Sí
Revisar solicitudes de incorporación	No como función propia	Sí
Proteger una rama mediante reglas	No como política alojada	Sí
Ejecutar automatizaciones alojadas	No por sí solo	Sí

Instalación y comprobación de Git

Git está disponible para los principales sistemas operativos. Debe instalarse desde los canales oficiales del sistema o desde el sitio oficial de Git (**https://git-scm.com/**).

Una vez instalado, la disponibilidad se comprueba desde un terminal:

```
git --version
```

La respuesta muestra la versión instalada:

```
git version 2.x.x
```

El número exacto depende de la versión disponible. Lo importante es que la orden sea reconocida y Git responda correctamente.

Ayuda integrada

Git incorpora documentación accesible desde el terminal. La ayuda general se consulta mediante:

```
git help
```

Para abrir la documentación de una orden concreta:

```
git help commit
```

También puede utilizarse:

```
git commit --help
```

La opción abreviada `-h` muestra una referencia más breve:

```
git commit -h
```

Las interfaces gráficas y las integraciones incluidas en los entornos de desarrollo pueden simplificar determinadas operaciones. Sin embargo, siguen ejecutando conceptos y procesos de Git. Comprender las órdenes fundamentales permite saber qué está ocurriendo y resolver incidencias que una interfaz gráfica puede ocultar.

Configuración de la rama inicial

Para establecer `main` como nombre predeterminado de la rama inicial de los nuevos repositorios:

```
git config --global init.defaultBranch main
```

También puede indicarse al crear un repositorio:

```
git init -b main
```

Importante: La documentación y los repositorios existentes pueden utilizar `main`, `master` u otro nombre. El nombre no modifica el funcionamiento esencial de la rama. En este capítulo se utilizará `main`.

Advertencia: No deben ejecutarse órdenes copiadas de Internet sin comprender su efecto. Algunas pueden modificar el directorio de trabajo o eliminar cambios no registrados.

Identidad y configuración inicial

Cada confirmación registra una identidad de autor. Git necesita conocer un nombre y una dirección de correo:

```
git config --global user.name "Ana Martín"

git config --global user.email "ana.martin@example.com"
```

La opción `--global` aplica la configuración al usuario del sistema. Si se omite dentro de un repositorio, la opción solo se aplica a ese proyecto:

```
git config user.email "ana.proyecto@example.com"
```

La configuración local tiene prioridad sobre la global. Esto permite utilizar identidades diferentes en proyectos distintos.

Para consultar toda la configuración visible:

```
git config --list
```

Para consultar un valor concreto:

```
git config user.name
```

Para conocer el archivo del que procede cada valor:

```
git config --list --show-origin
```

Niveles de configuración

Sistema

Se aplica a todas las cuentas del equipo.

Global

Se aplica a la cuenta actual.

Local

Se aplica únicamente al repositorio activo.

Git combina los tres niveles. Cuando una opción aparece en varios, prevalece el nivel más específico.

Identidad y autenticación

El nombre y el correo configurados se incorporan a las confirmaciones, pero no sirven para iniciar sesión en GitHub. La autenticación con la plataforma se configura por separado mediante HTTPS o SSH.

Para que GitHub asocie una confirmación a una cuenta, la dirección registrada debe estar vinculada a esa cuenta o debe emplearse la dirección privada proporcionada por GitHub.

Importante: La autoría escrita en una confirmación y la comprobación criptográfica de una identidad son cuestiones diferentes. Git también permite firmar confirmaciones y etiquetas, pero ese mecanismo no es necesario para comprender el flujo básico desarrollado aquí.

Crear o clonar un repositorio

Un repositorio puede obtenerse inicializando un proyecto local existente o clonando un repositorio creado previamente.

Inicializar un proyecto local

Desde la carpeta de AulaDeploy:

```
cd auladeploy

git init -b main
```

Git crea un subdirectorio oculto denominado `.git`. En él almacena los objetos, las referencias, la configuración local y los demás datos que forman el repositorio.

La carpeta de trabajo continúa conteniendo los archivos ordinarios. La existencia de `.git` es lo que permite que Git interprete esa carpeta como un repositorio.

Clonar un repositorio existente

Cuando AulaDeploy ya está alojado en GitHub:

```
git clone
https://github.com/organizacion/auladepl
oy.git
```

Git crea una carpeta, descarga los archivos, obtiene el historial y configura un remoto llamado `origin`.

Puede indicarse un nombre diferente para la carpeta local:

```
git clone
https://github.com/organizacion/auladepl
oy.git portal
```

Diferencia esencial

git init

Crea un repositorio nuevo a partir de una carpeta local.

git clone

Crea una copia local completa de un repositorio existente y configura su relación inicial con el remoto.

⚠ **Advertencia:** Eliminar `.git` no borra los archivos del proyecto, pero elimina el historial, las ramas y la configuración local del repositorio. Su contenido no debe modificarse manualmente sin conocer con precisión la estructura interna de Git.
No debe ejecutarse `git init` dentro de una copia clonada. La clonación ya crea y configura el repositorio local.

Las tres zonas de trabajo

Para comprender el funcionamiento cotidiano de Git es necesario distinguir tres zonas.

Directorio de trabajo

Contiene los archivos visibles y editables. Modificar `index.html` cambia el directorio de trabajo, pero no registra todavía una versión.

Área de preparación

También se denomina índice o *staging area*. Permite seleccionar el contenido exacto que formará parte de la siguiente confirmación. Representa una propuesta concreta para el próximo estado que se registrará.

Repositorio local

Contiene el historial confirmado. Una confirmación registra una instantánea coherente del contenido preparado y la relaciona con la historia anterior.

Flujo básico



Importante: `git add` copia al área de preparación el estado actual del archivo. Si el archivo vuelve a modificarse después, la nueva modificación no queda preparada automáticamente. Por ello, un mismo archivo puede contener simultáneamente una parte preparada y otra pendiente.
La existencia del área de preparación permite decidir qué cambios se registran juntos. No es obligatorio incluir en una misma confirmación todo lo que se haya modificado en el directorio de trabajo.

Estados de los archivos

Git clasifica los archivos según su relación con el historial y el área de preparación.

Sin seguimiento

El archivo existe en el directorio de trabajo, pero Git todavía no lo ha incorporado al seguimiento.

Sin modificar

El archivo coincide con el estado registrado en la confirmación actual.

Modificado

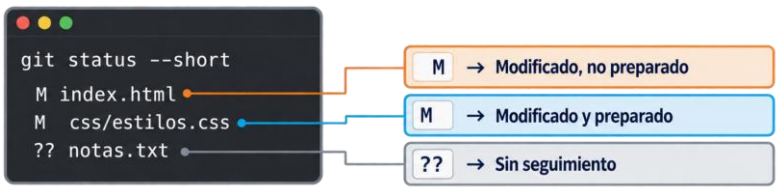
El archivo está siendo seguido, pero su contenido difiere del último estado confirmado.

Preparado

El estado actual del archivo se encuentra en el área de preparación y formará parte de la siguiente confirmación.

El ciclo habitual es:





Consulta del estado

```
git status
```

Esta orden explica qué archivos han cambiado y qué acciones pueden realizarse.

Para obtener una salida abreviada:

```
git status --short
```

Ejemplo

```
 M index.html
M  css/estilos.css
?? notas.txt
```

La posición de las letras es significativa: `M` indica una modificación no preparada. `M` indica una modificación preparada. `??` indica un archivo sin seguimiento.

Importante: `git status` no modifica el repositorio. Es una orden de consulta y conviene utilizarla antes y después de cualquier operación relevante.

Antes de continuar: Si un archivo se prepara con `git add` y después vuelve a modificarse, la siguiente confirmación incorporará el estado preparado, no necesariamente la última modificación realizada.

Seleccionar cambios con git add

Para preparar un archivo concreto:

```
git add index.html
```

Para preparar varios archivos:

```
git add index.html css/estilos.css
```

Para preparar los cambios situados en la carpeta actual y sus subcarpetas:

```
git add .
```

Para preparar archivos nuevos, modificaciones y eliminaciones del repositorio:

```
git add -A
```

Después debe comprobarse el resultado:

```
git status
```

El área de preparación permite construir confirmaciones coherentes. Si se han resuelto dos problemas independientes, no es obligatorio registrarlos juntos. Puede prepararse primero el cambio relacionado con el primer problema, confirmarlo y registrar después el segundo.

Preparación interactiva

```
git add -p
```

Esta orden muestra los bloques modificados y permite decidir cuáles deben incorporarse al área de preparación.

`index.html` contiene una corrección ortográfica y una modificación del menú. Si ambas responden a tareas diferentes, la preparación interactiva permite registrar cada una en una confirmación distinta.

Secuencia de revisión

```
git status
git diff
git add <archivo>
git diff --staged
git commit
```

⚠ **Importante:** Preparar todos los archivos sin revisarlos puede incorporar registros, credenciales, archivos temporales o cambios sin relación. El área de preparación debe utilizarse como una selección deliberada, no como un trámite automático.

Crear confirmaciones coherentes

Una confirmación, denominada habitualmente *commit*, registra el contenido del área de preparación y lo incorpora al historial.

```
git commit -m "Corrige la validación del formulario"
```

Cada confirmación contiene, entre otros datos: un identificador, la autoría, la fecha, un mensaje, la instantánea de los archivos y una referencia a la confirmación anterior, salvo en la primera.

El identificador se calcula criptográficamente a partir del contenido y de otros datos. Habitualmente se muestra de forma abreviada.

Una confirmación no es

- Una simple copia manual.
- Una sincronización automática con GitHub.
- Una publicación en un servidor web.
- Un registro de todos los cambios existentes, sino de los preparados.
- Un sustituto de una explicación comprensible.

Características de una buena confirmación



Atómica

Responde a un propósito reconocible.



Coherente

Todos sus cambios guardan relación.



Comprensible

El mensaje explica el resultado conseguido.



Comprobada

El proyecto sigue funcionando en el estado registrado.



Pequeña, pero completa

No mezcla tareas independientes ni divide artificialmente una modificación inseparable.

Ejemplos

Adecuado: `Corrige la validación del correo en el registro`

Poco informativo: `Cambios`

Demasiado amplio: `Actualiza menú, corrige formulario, cambia colores y elimina pruebas`

> **Importante:** El historial constituye una narración técnica de la evolución del proyecto. Un historial claro facilita la revisión, el diagnóstico, la recuperación y la comprensión del código.

Consultar diferencias e historial

Git permite examinar el trabajo antes y después de registrarlo.

Cambios no preparados

```
git diff
```

Compara el directorio de trabajo con el área de preparación.

Cambios preparados

```
git diff --staged
```

Compara el área de preparación con la última confirmación.

Historial completo

```
git log
```

Historial compacto

```
git log --oneline
```

Historial con ramas

```
git log --oneline --graph --decorate --all
```

Detalle de una confirmación

```
git show <identificador>
```

Comparación entre referencias

```
git diff <referencia1> <referencia2>
```

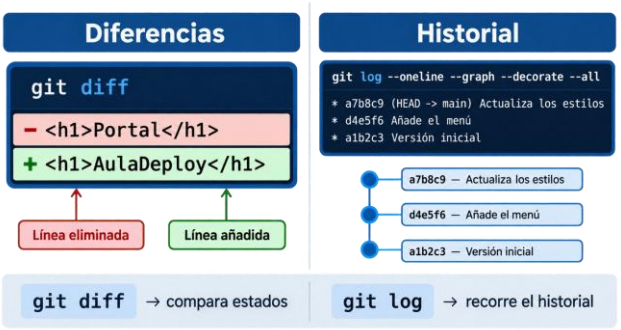
Las referencias pueden ser identificadores, ramas o etiquetas.

Interpretación de una diferencia

Las líneas precedidas por `-` se han eliminado respecto al estado comparado. Las líneas precedidas por `+` se han añadido. Estos caracteres pertenecen a la representación de la diferencia; no forman parte del archivo.

```
-<h1>Portal</h1>
+<h1>AulaDeploy</h1>
```

La línea anterior se ha sustituido por una nueva.



Importante: `git log` permite recorrer el historial. `git diff` permite comparar estados. Ninguna de estas órdenes modifica archivos.

Antes de continuar: Si `git diff` no muestra nada, aún podrían existir cambios preparados. En ese caso deben consultarse mediante `git diff --staged`.

Archivos que no deben versionarse

No todos los archivos de un proyecto deben incorporarse al repositorio. Normalmente se excluyen:

- Dependencias reconstruibles.
- Resultados de compilación.
- Archivos temporales.
- Registros de ejecución.
- Cachés.
- Configuraciones personales del entorno.
- Credenciales y secretos.
- Copias locales de bases de datos con información real.

El archivo `.gitignore` contiene patrones que indican qué archivos sin seguimiento debe ignorar Git.

Ejemplo

```
.env
logs/
*.log
node_modules/
dist/
.idea/
.vscode/
```

Los patrones deben adaptarse al proyecto. Excluir un archivo no significa que carezca de importancia, sino que su contenido no debe almacenarse en el repositorio.

Configuración de ejemplo

Puede versionarse una plantilla sin información sensible:

```
config/app.example.json
```

La configuración real queda ignorada:

```
config/app.json
```

⚠ **Advertencia:** `.gitignore` no deja de seguir un archivo que ya fue confirmado. Si `config/app.json` forma parte del historial, añadirlo a `.gitignore` no lo elimina del seguimiento.
Para retirarlo del índice y conservar la copia local:

```
git rm --cached config/app.json
```

Después debe confirmarse el cambio.

Si el archivo contenía un secreto, retirarlo del estado actual tampoco elimina ese dato de las confirmaciones anteriores. La credencial debe considerarse expuesta y renovarse.

Corregir cambios sin perder el control

La operación adecuada depende de dónde se encuentre el cambio y de si ya se ha compartido.

Descartar cambios no preparados

```
git restore index.html
```

Restaura el archivo desde el índice. Los cambios locales afectados se pierden.

Retirar del área de preparación

```
git restore --staged index.html
```

El archivo continúa modificado en el directorio de trabajo, pero deja de formar parte de la próxima confirmación.

Corregir la última confirmación

```
git commit --amend
```

Permite sustituir la última confirmación, por ejemplo para añadir un archivo olvidado o corregir el mensaje. La confirmación resultante tiene un identificador diferente.

Revertir una confirmación compartida

```
git revert <identificador>
```

Crea una nueva confirmación que aplica el cambio inverso. El historial conserva tanto el cambio original como su reversión.

Mover una referencia

`git reset` puede desplazar la rama actual y modificar, dependiendo del modo, el índice y el directorio de trabajo.

⚠ **Advertencia:** `git reset --hard` puede eliminar cambios no confirmados. No debe utilizarse como solución automática.

Criterio general

- Cambio local no preparado: `git restore`.
- Cambio preparado por error: `git restore --staged`.
- Última confirmación local no compartida: `git commit --amend`.
- Confirmación ya compartida: normalmente `git revert`.

Antes de corregir debe determinarse qué estado se desea conservar. Ejecutar órdenes sin esa decisión puede agravar el problema.

Etiquetas y versiones reconocibles

Las confirmaciones se identifican mediante cadenas criptográficas, pero esos identificadores no siempre resultan adecuados para señalar versiones publicadas. Una etiqueta proporciona un nombre estable a una confirmación concreta.

Crear una etiqueta anotada

```
git tag -a v1.0.0 -m "Primera versión estable"
```

Consultar etiquetas

```
git tag
```

Mostrar una etiqueta

```
git show v1.0.0
```

Publicar una etiqueta

```
git push origin v1.0.0
```

Publicar todas las etiquetas

```
git push origin --tags
```

Las etiquetas no se publican necesariamente como consecuencia de un envío ordinario. Deben transferirse explícitamente cuando corresponda.

Rama y etiqueta

Una rama es una referencia móvil. Avanza cuando se crean nuevas confirmaciones sobre ella.

Una etiqueta suele permanecer asociada a la misma confirmación y señala un hito concreto.

Versionado semántico

Una convención habitual emplea tres números:

MAYOR.MENOR.PARCHE

- **MAYOR** identifica cambios incompatibles.
- **MENOR** identifica funciones nuevas compatibles.
- **PARCHE** identifica correcciones compatibles.

Git no impone esta convención. Su utilización depende de las normas adoptadas por el proyecto.

`v1.2.3` puede representar la tercera corrección de la versión menor 1.2.

> **Importante:** Una etiqueta identifica el código correspondiente a una versión. Una publicación puede incluir además notas, documentación y artefactos preparados para su distribución.

Qué es una rama

Una rama es una referencia que señala una confirmación. Cuando se crean nuevas confirmaciones sobre ella, la referencia avanza.

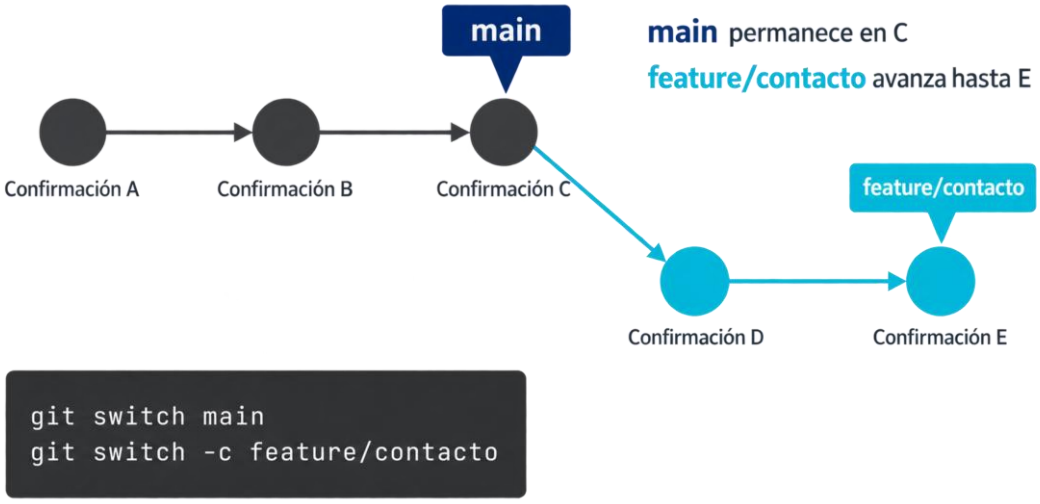
Las ramas permiten mantener líneas de trabajo diferenciadas sin duplicar manualmente el proyecto. Una modificación puede desarrollarse en una rama independiente y permanecer separada de `main` hasta que se encuentre lista para integrarse.

Supuesto

La rama `main` contiene una versión estable. Es necesario añadir un formulario de contacto:

```
git switch -c feature/contacto
```

La nueva rama comienza en la misma confirmación que `main`. Las confirmaciones creadas posteriormente hacen avanzar `feature/contacto`, mientras `main` permanece en su estado anterior.



Operaciones con ramas

```
git branch
```

```
git branch feature/contacto
```

```
git switch feature/contacto
```

```
git switch -c feature/contacto
```

```
git switch main
```

```
git branch -d feature/contacto
```

Importante: Una rama no contiene una copia independiente de todos los archivos. Es una referencia ligera dentro del historial. Al cambiar de rama, Git ajusta el directorio de trabajo al estado señalado por la nueva referencia.

Integrar cambios mediante merge

Cuando el trabajo de una rama está terminado, puede integrarse en otra. Para incorporar `feature/contacto` a `main`:

```
git switch main

git merge feature/contacto
```

La fusión se ejecuta desde la rama que recibirá los cambios.

Avance rápido

Si `main` no ha recibido nuevas confirmaciones desde que se creó la rama, Git puede mover directamente la referencia de `main` hasta la confirmación más reciente de `feature/contacto`.

Confirmación de fusión

Si ambas ramas han avanzado, Git combina sus historiales y puede crear una confirmación de fusión con dos antecedentes.

Antes de integrar

- Comprobar el estado.
- Confirmar o guardar los cambios locales.
- Revisar las diferencias.
- Ejecutar las pruebas.
- Verificar que la rama responde a un propósito reconocible.
- Comprobar que no contiene archivos innecesarios o secretos.

Secuencia

```
git switch main

git merge feature/contacto

git branch -d feature/contacto
```

Eliminar la rama después de integrarla no elimina las confirmaciones. Estas ya forman parte de la historia alcanzable desde `main`.

Importante: Git puede combinar historiales, pero no determina si una función satisface los requisitos o si el resultado es correcto. La revisión y las pruebas deben realizarse antes y después de la integración.

Conflictos de fusión

Git combina automáticamente cambios realizados en archivos diferentes o en zonas independientes. Un conflicto aparece cuando no puede decidir de forma segura cómo construir el resultado.

Situación

En main:

```
<h1>Portal de despliegue</h1>
```

En feature/portada:

```
<h1>AulaDeploy</h1>
```

Si ambas ramas modifican la misma línea de forma incompatible, Git detiene la fusión y añade marcadores:

```
<<<<<<< HEAD
<h1>Portal de despliegue</h1>
=======
<h1>AulaDeploy</h1>
>>>>>>> feature/portada
```

La parte superior representa la rama actual. La parte inferior procede de la rama que se está integrando.

Proceso de resolución

01

Abrir el archivo y comprender el propósito de los dos cambios

02

Elegir una versión o construir una combinación válida y eliminar los marcadores

03

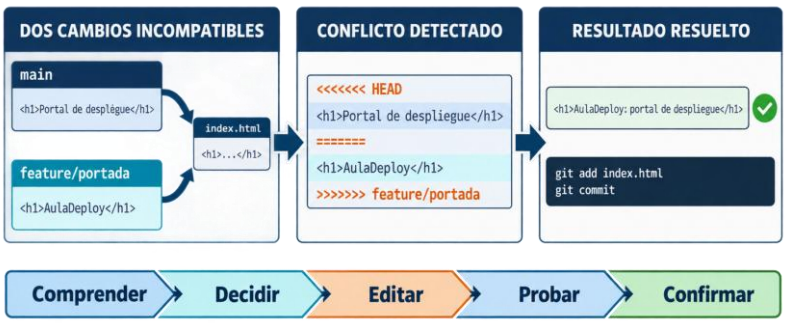
Guardar, comprobar la sintaxis y probar el funcionamiento

04

Preparar el archivo y completar la confirmación

```
git add index.html

git commit
```



Cancelar la fusión

```
git merge --abort
```

Esta orden intenta devolver el repositorio al estado anterior al inicio de la fusión.

⚠ **Importante:** Resolver un conflicto no consiste únicamente en borrar los marcadores. El resultado debe ser correcto y respetar la intención de los cambios implicados.

Repositorios remotos

Un remoto es una referencia configurada hacia otro repositorio. Permite intercambiar confirmaciones, ramas y etiquetas.

Consultar remotos

```
git remote -v
```

Añadir un remoto

```
git remote add origin
https://github.com/organizacion/auladeploy
.git
```

Consultar información

```
git remote show origin
```

Cambiar la dirección

```
git remote set-url origin <nueva-dirección>
```

`origin` es un nombre convencional. Git lo asigna habitualmente al remoto principal durante la clonación, pero puede utilizarse otro nombre.

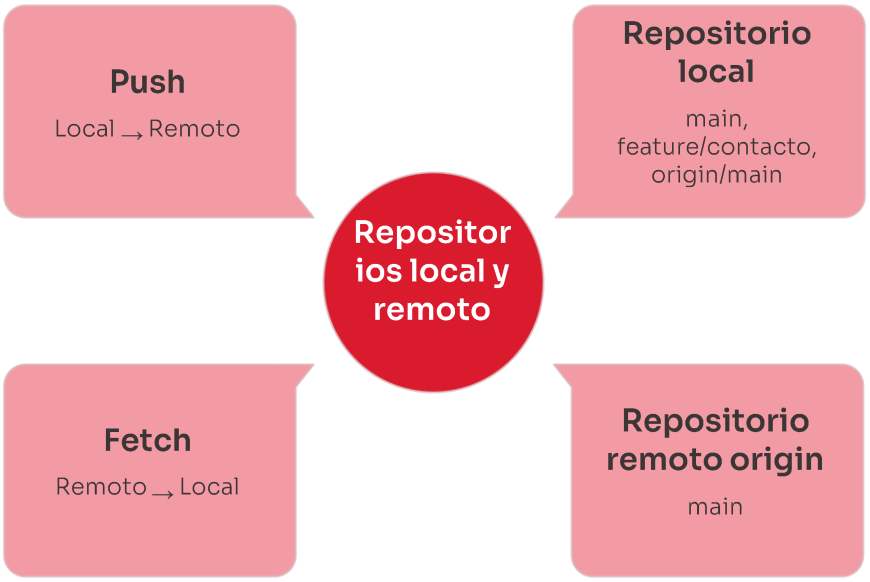
Referencias remotas

Una referencia como `origin/main` representa el último estado conocido de la rama `main` del remoto `origin`.

No es la propia rama situada en el servidor ni se actualiza continuamente. Cambia cuando se obtiene información del remoto.

El repositorio local puede contener: ramas locales, confirmaciones todavía no publicadas, referencias remotas, etiquetas y configuración propia.

La existencia de un remoto no implica sincronización automática. Deben ejecutarse operaciones de obtención o publicación.



Fetch, pull y push

Las órdenes `fetch`, `pull` y `push` intervienen en la sincronización, pero no realizan la misma operación.

fetch

```
git fetch origin
```

Descarga información y actualiza las referencias remotas. No integra automáticamente los cambios en la rama de trabajo.

pull

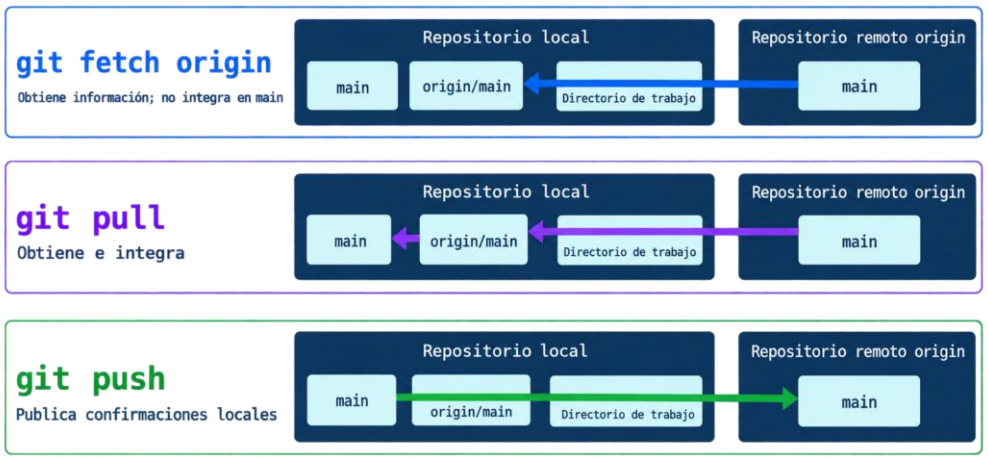
```
git pull
```

Obtiene cambios y los integra en la rama actual según la configuración establecida. La integración puede realizarse mediante fusión o rebase.

push

```
git push
```

Envía confirmaciones al remoto y actualiza la rama remota si la operación está permitida.



Primera publicación de main

```
git push -u origin main
```

La opción `-u` establece una relación de seguimiento entre la rama local y la remota.

Publicar una rama de trabajo

```
git push -u origin feature/contacto
```

Después pueden utilizarse `git push` y `git pull` sin indicar siempre el remoto y la rama.

Diferencia esencial

- `fetch` obtiene información y permite examinarla antes de integrar.
- `pull` obtiene e integra.
- `push` publica cambios locales.

Rechazo de una publicación

El remoto puede rechazar un push si contiene cambios que la rama local no ha integrado, si la rama está protegida o si no existen permisos suficientes.

⚠ **Advertencia:** Forzar un envío puede sobrescribir referencias remotas y dificultar la recuperación del trabajo compartido. No debe utilizarse como respuesta automática ante un rechazo.

Autenticación con GitHub

La identidad configurada mediante `user.name` y `user.email` no constituye una credencial de acceso. Cuando una operación remota requiere autenticación, debe utilizarse un mecanismo admitido por GitHub.

HTTPS

Una dirección HTTPS presenta esta forma:

```
https://github.com/organizacion/auladeploy.git
```

GitHub no admite la contraseña ordinaria de la cuenta para autenticar operaciones Git por HTTPS. La autenticación puede gestionarse mediante una herramienta de credenciales compatible o mediante un token cuando el procedimiento lo requiera.

Un token debe:

- Tener el alcance mínimo necesario.
- Disponer de una caducidad adecuada.
- Guardarse mediante un mecanismo seguro.
- Renovarse si se expone.
- Permanecer fuera de los archivos versionados.

SSH

Una dirección SSH presenta esta forma:

```
git@github.com:organizacion/auladeploy.git
```

SSH utiliza un par de claves:

- **Clave privada.** Permanece protegida en el equipo.
- **Clave pública.** Se registra en GitHub.

La clave privada no debe compartirse ni incorporarse al repositorio. Puede protegerse mediante una frase de paso.

Autenticación multifactor

La autenticación multifactor añade una comprobación al acceso a la cuenta. No sustituye la custodia de tokens y claves.

Advertencia: Si un token o una clave privada aparecen en una confirmación, deben considerarse comprometidos aunque el repositorio sea privado o el archivo se borre posteriormente. La credencial debe revocarse o renovarse.

Repositorios, visibilidad y permisos

Un repositorio de GitHub combina el contenido gestionado por Git con servicios de administración y colaboración.

Visibilidad

Repositorio público. Su contenido puede consultarse públicamente, aunque la capacidad de modificarlo depende de los permisos.

Repositorio privado. El acceso queda limitado a las identidades autorizadas.

La visibilidad privada no constituye una protección adecuada para los secretos. Un repositorio privado puede clonarse, compartirse incorrectamente o quedar expuesto.

Permisos

Los permisos determinan qué operaciones puede realizar cada identidad. Debe aplicarse el principio de mínimo privilegio: conceder únicamente las capacidades necesarias.

Archivos habituales

README.md

Presenta el propósito, los requisitos, la instalación y el uso.

LICENSE

Establece las condiciones jurídicas de reutilización.

CONTRIBUTING.md

Explica cómo proponer y revisar cambios.

.gitignore

Define patrones de archivos que no deben versionarse.

Medidas de seguridad

- Restringir los permisos administrativos.
- Proteger la rama principal.
- Revisar las incorporaciones.
- Evitar credenciales en los archivos.
- Controlar las integraciones externas.
- Revisar periódicamente colaboradores y claves.
- Mantener las dependencias actualizadas.

Importante: Tener permiso técnico para escribir en un repositorio no implica que cualquier cambio deba incorporarse directamente a la rama principal. Los permisos y el proceso de revisión cumplen funciones complementarias.

Incidencias y solicitudes de incorporación

Una incidencia, denominada *issue* en GitHub, describe un error, una necesidad, una mejora o una tarea. Debe aportar la información necesaria para comprender el problema y comprobar su resolución.

Ejemplo de incidencia

Título: «El formulario acepta direcciones sin dominio»

Contenido:

- Situación observada.
- Resultado esperado.
- Pasos para reproducirla.
- Criterios de aceptación.
- Información técnica relevante.

Una solicitud de incorporación, denominada *pull request*, propone integrar los cambios de una rama en otra. No es únicamente una petición de fusión. También proporciona un espacio para examinar diferencias, discutir decisiones, ejecutar comprobaciones y aprobar o solicitar modificaciones.

Una revisión debe comprobar

- Si el cambio resuelve la necesidad.
- Si incorpora archivos innecesarios.
- Si introduce riesgos.
- Si mantiene la legibilidad.
- Si las pruebas son suficientes.
- Si la documentación continúa siendo válida.

Flujo de trabajo



Importante: Revisar no consiste únicamente en buscar errores sintácticos. También implica valorar seguridad, claridad, mantenibilidad y correspondencia con el propósito declarado.

Protección de la rama principal

La rama principal suele representar el estado estable o integrable del proyecto. Permitir modificaciones directas sin comprobación puede dejarla en una situación inconsistente.

Las reglas de protección pueden exigir condiciones como:

- Utilizar una solicitud de incorporación.
- Obtener un número mínimo de aprobaciones.
- Resolver conversaciones pendientes.
- Superar comprobaciones automáticas.
- Mantener la rama actualizada.
- Restringir quién puede publicar.
- Impedir eliminaciones.
- Impedir envíos forzados.

Ejemplo de política

Para la rama `main` se establece:

1. No se permite publicar directamente.
2. Todo cambio llega mediante una solicitud de incorporación.
3. Se requiere al menos una aprobación.
4. Las comprobaciones deben finalizar correctamente.
5. No se permiten envíos forzados.

Aunque una identidad disponga de permiso de escritura, deberá seguir el flujo establecido. De este modo, las normas de trabajo dejan de ser simples recomendaciones y se convierten en controles aplicados por la plataforma.

Limitaciones

Una regla técnica no garantiza por sí sola la calidad. Una aprobación superficial o una prueba insuficiente pueden permitir un cambio defectuoso. Las reglas complementan el análisis profesional, pero no lo sustituyen.

Antes de continuar: Si un push es rechazado por una rama protegida, la solución no consiste en forzar la operación. Debe identificarse la condición pendiente y seguir el flujo autorizado.

Secretos e incidentes de exposición

Un secreto es un dato que permite acceder a un recurso o realizar una operación privilegiada: contraseñas, tokens, claves privadas, credenciales de bases de datos o claves de servicios externos.

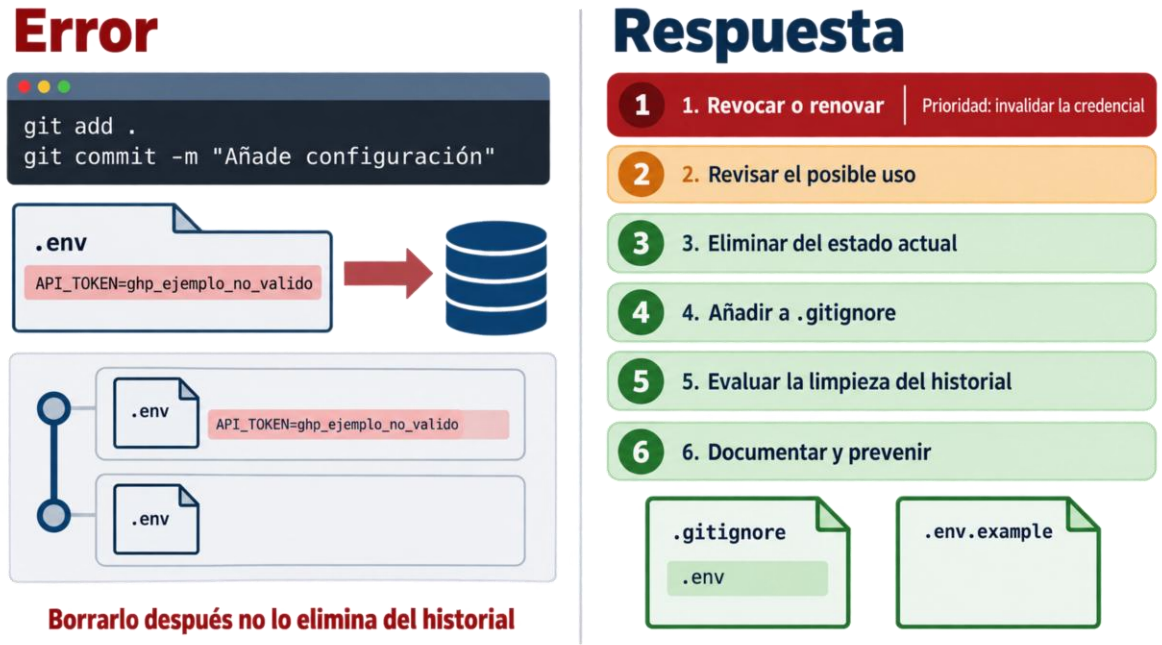
Los secretos no deben incluirse en el código ni en archivos versionados. Deben proporcionarse mediante variables de entorno, almacenes de secretos o archivos externos protegidos.

Supuesto

Se confirma por error:

```
API_TOKEN=ghp_ejemplo_no_valido
```

Borrar la línea y crear otra confirmación no resuelve completamente el problema. El valor puede continuar presente en el historial.



Prevención

- Revisar `git diff --staged`.
- Separar configuración y código.
- Limitar permisos y caducidad.
- Revisar antes de publicar.
- Utilizar `.gitignore`.
- Versionar plantillas sin valores reales.
- Utilizar detectores de secretos.

Respuesta adecuada

⚠ **Advertencia:** Limpiar el historial reescribe confirmaciones y puede afectar a todas las copias del repositorio. Debe planificarse y coordinarse. La prioridad es invalidar la credencial, porque modificar el historial no garantiza que nadie la haya copiado.

Flujo profesional completo

El control de versiones resulta más útil cuando las operaciones forman un proceso coherente.



Importante: El flujo puede adaptarse a las necesidades del proyecto, pero debe conservar la trazabilidad, la revisión, la seguridad y la capacidad de recuperación.

Errores frecuentes y diagnóstico

A continuación se describen los mensajes de error más habituales y su significado.

not a git repository

La orden se ha ejecutado fuera de un repositorio o Git no puede localizar `.git`. Debe comprobarse la ruta actual.

nothing to commit

No existen cambios preparados diferentes de la confirmación actual. Puede haber archivos ignorados o modificaciones todavía no guardadas por el editor.

your branch is ahead of...

La rama local contiene confirmaciones que todavía no se han publicado.

your branch is behind...

La rama remota contiene confirmaciones que la rama local todavía no ha integrado.

non-fast-forward

La actualización remota no puede realizarse directamente porque existen cambios que deben obtenerse e integrarse.

merge conflict

Git no puede combinar determinados cambios automáticamente. Deben resolverse los archivos afectados y completar o cancelar la fusión.

permission denied / authentication failed

La identidad no se ha autenticado correctamente o carece de permisos. Deben revisarse la dirección remota, el mecanismo de autenticación y la autorización.

protected branch

La plataforma impide una actualización directa. Debe seguirse el flujo exigido por las reglas de la rama.

Método de diagnóstico

1. Leer el mensaje completo.
2. Ejecutar `git status`.
3. Identificar la rama actual.
4. Determinar si el problema es local, remoto o de permisos.
5. Decidir qué estado debe conservarse.
6. Elegir una operación cuyo efecto se comprenda.
7. Comprobar el resultado.

Síntesis y glosario

Síntesis

Git registra la evolución de un conjunto de archivos mediante confirmaciones relacionadas. El trabajo se desarrolla en tres zonas: directorio de trabajo, área de preparación y repositorio. `git add` selecciona el contenido de la siguiente confirmación; `git commit` lo registra; y `git status`, `git diff` y `git log` permiten inspeccionar los distintos estados.

Las ramas aíslan líneas de trabajo y las fusiones permiten integrarlas. Cuando existen cambios incompatibles, el conflicto debe resolverse comprendiendo la intención de las dos versiones.

Los repositorios remotos permiten compartir el historial. `fetch` obtiene información, `pull` obtiene e integra y `push` publica cambios. GitHub aloja repositorios y añade incidencias, solicitudes de incorporación, permisos, revisión y protección de ramas.

La seguridad exige excluir credenciales, limitar permisos y reaccionar ante cualquier exposición renovando primero el secreto. Un historial claro, unas confirmaciones coherentes y un proceso de revisión convierten el repositorio en una fuente fiable sobre la evolución del proyecto.

Glosario

Área de preparación	Estado intermedio que contiene lo seleccionado para la siguiente confirmación.
Clon	Copia local de un repositorio con su historial.
Confirmación	Instantánea registrada y relacionada con la historia anterior.
Conflicto	Situación en la que Git no puede combinar cambios automáticamente.
HEAD	Referencia al estado o rama seleccionados actualmente.
Historial	Conjunto relacionado de confirmaciones.
Rama	Referencia móvil a una confirmación.
Remoto	Repositorio configurado para intercambiar cambios.
Repositorio	Estructura que conserva objetos, referencias, configuración e historial.
Solicitud de incorporación	Propuesta revisable para integrar una rama en otra.
Etiqueta	Referencia estable utilizada para identificar un hito.

Fuentes de información

Documentación oficial de Git

Referencia oficial de órdenes

Git. https://git-scm.com/docs

Pro Git, segunda edición. Acerca del control de versiones

Git. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Pro Git. Fundamentos de Git: registro de cambios

Git. https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository

Pro Git. Consulta del historial de confirmaciones

Git. https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History

Pro Git. Deshacer cambios

Git. https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things

Pro Git. Trabajo con repositorios remotos

Git. https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes

Pro Git. Etiquetado

Git. https://git-scm.com/book/en/v2/Git-Basics-Tagging

Pro Git. Ramas en pocas palabras

Git. https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Pro Git. Creación de ramas y fusiones

Git. https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

Referencia oficial de patrones de exclusión

Git. https://git-scm.com/docs/gitignore

Documentación oficial de GitHub

¿Qué es GitHub?

GitHub Docs. https://docs.github.com/en/get-started/start-your-journey/what-is-github

Autenticación en GitHub

GitHub Docs. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github

Conexión a GitHub mediante SSH

GitHub Docs. https://docs.github.com/en/authentication/connecting-to-github-with-ssh

Información general sobre las solicitudes de incorporación

GitHub Docs. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-pull-requests

Ramas protegidas

GitHub Docs. https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

Eliminación de datos confidenciales de un repositorio

GitHub Docs. https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/removing-sensitive-data-from-a-repository

Administración de repositorios y permisos

GitHub Docs. https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features